

The Sisyphean Struggle with Data Integrity: Recurrent dirty read and write

Introduction

GoShipment faced repeated data losses, like a never-ending struggle. It's similar to the ancient tale of Sisyphus pushing a boulder uphill, only to see it roll down again. In our story, the boulder is crucial data, and technical issues keep causing setbacks. Let's explore the reasons and find ways to stop the digital boulder from slipping away again.

The Saga

First Contact at Foothills

Around July of 2023, a cheeky [dirty write](#) was reported on the JSONB Column `goshipment_data`. Where multiple Sidekiq jobs that were updating this column were queued at the same time causing dirty writing issues with the column. This was a warning for an ugly winter of dirty read-and-write issues that were about to blizzard on us.

Issue 1 - Inappropriate Use of JSONB Columns: The burden of absence of DB Normalization

You ask why there were multiple sidekiq jobs queued, which were updating the same column at the same time. Good question!

That's because of this convenient thing called [ActiveRecord Callbacks](#), convenient but somewhat deadly.

As these callbacks were on a Model `Dispatch`, every time dispatch was saved. A callback was called which triggered those Sidekiq jobs.

Issue 2 - Inappropriate use of ActiveRecord Callbacks: The bombardment of Sidekiq

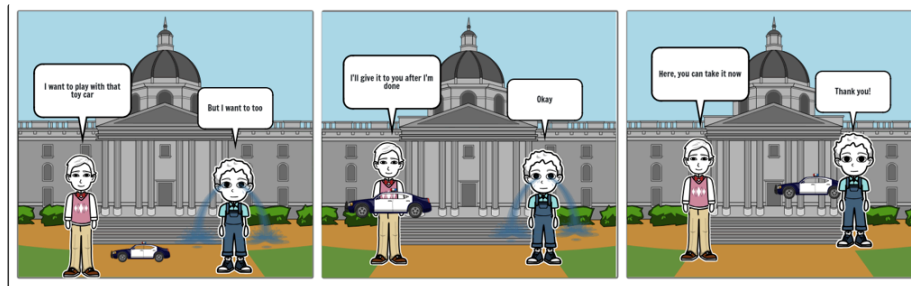
For that time these Sidekiq calls were moved from ActiveRecord Callbacks to directly at the places where these side-effects were needed.

Still, how did we solve **Issue 1**? Again a good question.

Birth of Application Level Mutex Locks

What is that?

Let's picture requests or APIs as kids sharing toys in a playground. Sometimes, multiple programs want the same toy simultaneously, leading to chaos. "Application Level Mutex Locks" act like a system of taking turns. When a request wants a specific toy (toy being our record here), it uses a lock, signaling, "I'm using this now." This ensures only one request plays with the toy at any given time. Once done, the lock is removed, allowing the next program to take its turn. It's a smart way to prevent requests from clashing, like kids taking turns on a slide, making the application organized and peaceful, without fights over who gets to play with what.



`ApplicationMutex` did give us some control over how exclusively a record from our table can be read or written. So, did we solve the problem?

Fast forward to October of 2023.

We again started seeing a surge in dirty read and write issues being reported with another column called `workflow_data` which was only a problem in the `goshipment_data` column.

`workflow_data` has a nested JSONB which again has data of multiple checklists of a shipment. Now, these checklists can be updated at the same time.

Are we seeing a pattern again? That's **Issue 1** again in a different situation.

⚠ **Issue 3** - ApplicationMutex needs to wrap every scenario where the record is fetched and updated: Cognitive overload of patching all the holes in the boat

So, `ApplicationMutex` is only useful where each fetch and update of the record is wrapped in a block by the `ApplicationMutex`. So, that means we need to find all such cases

We carpet-bombed every fetch and update of goshipment records with `ApplicationMutex` and the problem was somewhat solved.

Until...

The December of 2023

Introducing another limitation of `ApplicationMutex`, [race conditions](#) among the requests waiting for locks to release.

That is a mouthful, basically what happens is that our `ApplicationMutex` uses Redis store to maintain the lock status of a record, and it polls the lock status of a record every 0.1 seconds when waiting for a lock to release.

So, when multiple requests are waiting for a lock to be released, a request that was queued later might acquire the lock first because of the desync of 0.1 seconds.

To visualize this consider the below example,

Imagine a library with librarian Riya. Alice and Bob both want to read a rare book badly.

Librarian Riya ordered the book they wanted to read and told them that the library could order only a single book.

Alice goes to the library every morning at 10 AM to ask the librarian if the book has arrived.

The librarian checks for the book every noon at 2 PM only even if it might've arrived early.

Bob goes to the library every evening at 5 PM to ask the librarian if the book has arrived.

Now, even though Alice wanted the book first and has been asking for it every morning because the librarian only checks for the book at 2 PM, Bob will get the book first. Which isn't really fair.

⚠ **Issue 4** - There can be Race Conditions between the requests that are waiting for a lock to be released: Not so fast and not that furious

The above issue was solved by implementing a Redis Queue. Where requests were completed on a first come first serve basis.

📌 The Conclusion

So far it must've been a lot to sit through this journey but it's not over yet. To make sure the above never happens again, we have and will be taking the following steps.

1. Sensible use of Model Callbacks:
 - a. Model callbacks shouldn't be used for expensive calls such as above.
2. Separation and Isolation of Data:
 - a. We need to make sure all types of different data which has high update frequency are well isolated in a normalized format.
3. Normalization of Database models:
 - a. We need to avoid substituting a good normalized DB structure with JSONB columns.